# Cloud Computing
# (UEC634)
# Project Report

**Submitted By-**
**Mehar**
**102495013**
**3O22**

**Submitted To-**
**Dr.Ajay Kakkar**

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING**
**THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY, (A DEEMED TO BE UNIVERSITY),**

**PATIALA, PUNJAB, INDIA**
**2026**

## 1) AIM:

To design and implement a secure cloud-based file management system that provides:

- User authentication and authorization (IAM)
- File upload and download via a web interface
- Automatic backup and cross-region replication
- Multiple file versions (V1, V2, …) using versioning
- Lifecycle policies for storage management
- Testing and deployment on cloud infrastructure

## 2)ABSTRACT:

Cloud computing has revolutionized data storage by offering scalable, reliable, and secure solutions. This project, focuses on building a secure cloud storage application using AWS services. The system provides user authentication, file upload and download functionality, automatic versioning, and replication-based backup. Amazon S3 is used as the storage backend, while an EC2 instance hosts the web application developed using Flask. Identity and Access Management (IAM) ensures controlled access to cloud resources. The solution demonstrates a real-world implementation of cloud storage with security, reliability, and high availability.

## 3) TOOL USED:

Cloud Platform

- AWS (Amazon Web Services)

AWS Services

- Amazon S3: Main storage bucket (upload/download)
- S3 Versioning: Maintains V1, V2… of files
- S3 Replication : Replicates data from main bucket to backup bucket
- S3 Lifecycle Policy: Auto-move old versions to cheaper storage / delete after time
- AWS IAM: Role-based secure access (LabRole)
- Amazon EC2 (IaaS): Hosting the web application server (Linux instance)

Development Tools

- PuTTY (SSH client)
- Python 3
- Flask (Python Web Framework)
- Boto3 (AWS SDK for Python) to access S3 securely
- HTML/CSS/Bootstrap for UI (web interface)

## 4)INTRODUCTION:

With the rapid growth of data and digital services, cloud computing has become an essential technology. Cloud storage allows users to store data remotely with high availability and security. AWS provides various services that enable developers to build scalable and secure applications. This project leverages AWS S3 for storage, EC2 for application hosting, and IAM for secure access control, delivering a fully functional cloud storage system.

## 5) THEORY:

### 5.1 Cloud IAM (Identity and Access Management)

In any cloud-based system, security is one of the most important requirements. Identity and Access Management (IAM) is a service that helps in controlling access to cloud resources by allowing only authorized users to perform specific actions. It ensures that the right person gets the right level of access to the right resources.

IAM mainly works on two important concepts:

- Authentication: It verifies the identity of a user by checking login credentials such as username, password, or security tokens.
- Authorization: After verifying identity, it decides what actions the user is allowed to perform, such as uploading, downloading, or viewing files.

In Amazon Web Services (AWS), IAM access is managed using:

- Users
- Groups
- Roles
- Policies

Policies are written in JSON format and define the permissions assigned to users or roles. AWS also follows the principle of Least Privilege, which means that a user or application is given only the minimum permissions required to perform its task.

In this project, IAM plays a crucial role by allowing the EC2 instance (which hosts the web application) to securely interact with the Amazon S3 bucket using an IAM Role (LabRole). It also helps in restricting actions such as file upload, download, file listing, and version viewing to only authorized users.

## 5.2 Amazon S3 (Simple Storage Service)

Amazon S3 is a cloud-based object storage service used to store and retrieve data from anywhere on the internet. It stores data in the form of containers called buckets, which further contain files known as objects.

Each object stored inside a bucket has:

- A unique key (file name or path)
- Size of the file
- Metadata information
- Storage class such as Standard, Infrequent Access (IA), or Glacier

In this project, Amazon S3 is used as the primary storage service to store user-uploaded files securely. The main bucket stores the original files uploaded by users through the web interface.

## 5.3 Versioning (V1, V2, …)

Versioning is a feature of Amazon S3 that allows multiple versions of the same file to be stored within a bucket. When versioning is enabled, uploading a file with the same name does not permanently overwrite the existing file. Instead, Amazon S3 creates a new version of the file and assigns it a unique Version ID.

This helps in:

- Preventing accidental file deletion or overwriting
- Recovering previous versions of a file when required
- Maintaining a history of file changes

For example, if a user uploads a file named "report.pdf" multiple times, S3 will store it as different versions such as V1, V2, V3, etc. This allows the user to retrieve any previous version if needed.

## 5.4 Replication (Backup and Replicate)

Replication is a process that automatically copies data from one S3 bucket to another bucket. In this project, replication is used to create a backup of files stored in the main bucket by copying them into a secondary backup bucket.

There are two types of replication in AWS:

- Same Region Replication (SRR): Copies files within the same region.

- Cross Region Replication (CRR): Copies files to a bucket located in a different geographical region.

Replication improves system reliability by:

- Providing data backup
- Supporting disaster recovery
- Ensuring data availability even if the main bucket fails

Whenever a user uploads a file into the main bucket, it is automatically replicated into the backup bucket, ensuring that data is always available in case of failure or loss.

## 5.5 Lifecycle Policy

Lifecycle policies in Amazon S3 are used to manage stored data automatically over time. These policies help in organizing data and reducing storage costs by performing predefined actions on objects after a certain period.

Lifecycle rules can:

- Move older versions of files to low-cost storage classes like Glacier
- Permanently delete files or older versions after a specified number of days

This automated data management helps in:

- Optimizing storage cost
- Maintaining storage efficiency
- Removing unnecessary outdated data

## 5.6 Web Interface (Cloud Application Layer)

A web interface is developed in this project to allow users to interact with the cloud storage system in an easy and user-friendly manner. Through this interface, users can:

- Log in securely
- Upload files
- Download files
- View the list of uploaded files
- Access different versions of files

The interface is connected to the Amazon S3 storage using AWS SDK. It ensures that all operations performed by the user are secure and authorized using IAM roles and policies.

This makes the developed system function similar to modern cloud storage applications like Google Drive, but with additional security features such as IAM-based access control, automatic backup through replication, and version management.

**6)ARCHITECTURE DESCRIPTION:**

- User accesses the web application via browser
- Login authentication is validated
- Authenticated user uploads file
- Flask app sends request to AWS S3
- S3 stores file with versioning enabled
- Replication copies file to backup bucket

**7) PROCEDURE:**

**Step 1: Create S3 Buckets**

1. Create Main bucket (meharproject)
2. Create Backup bucket (projectmehar)

**Step 2: Enable Versioning**

1. Open Main bucket → Properties → Versioning → Enable
2. Open Backup bucket → Properties → Versioning → Enable (required for replication)

**Step 3: Configure Replication**

1. In main bucket: Management → Replication rules → Create rule
2. Select destination bucket (backup bucket)
3. Choose IAM role (create new or use existing replication role)
4. Enable rule for all obje.

**Step 4: Add Lifecycle Policy**

1. Main bucket → Management → Lifecycle rules
2. Define:
    - Move old versions after X days (example 30 days)
    - Delete old versions after Y days (example 90 days)

**Step 5: Launch EC2 Instance (Hosting)**

1. Launch Linux EC2
2. Allow inbound:

    - SSH (22)
    - HTTP (80) / Custom port (5000 for Flask)
3. Attach LabRole so EC2 can access S3

**Step 6: Connect Using PuTTY / SSH**

1. Convert key to .ppk
2. Login to EC2
3. Update packages and install python & dependencies

**Step 7: Build Web Application**

Using Flask:

- /login for authentication
- /upload for file upload to main bucket
- /files to list uploaded files
- /download/<filename> to download
- /versions/<filename> to show all versions (V1, V2…)

**Step 8: Testing**

Test cases:

- Login correct/incorrect
- Upload file and check:
    - present in main bucket
    - replicated in backup bucket
- Upload same name again:
    - version increments
    - old version still accessible
- Download file and verify data integrity

**Step 9: Deployment**

- Run Flask app on EC2
- Access through public IP:
    - http://<EC2-Public-IP>:5000

## 8)SNAPSHOTS:

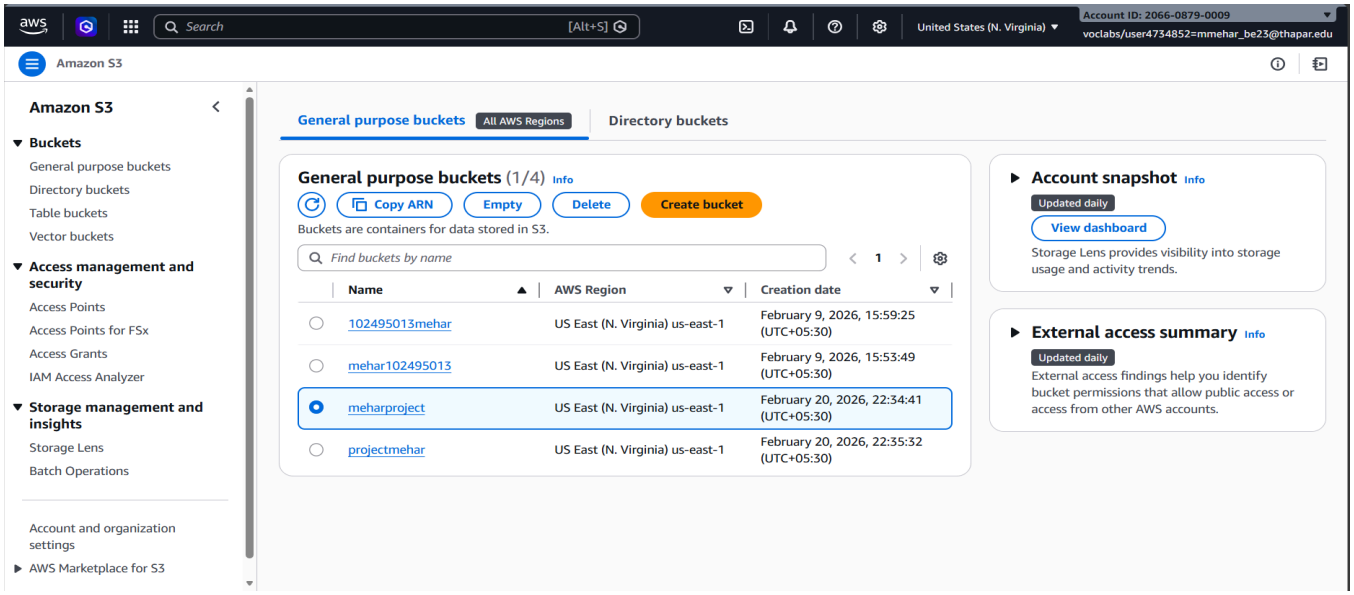


**Fig.1-S3 Bucket Setup**

```python
from flask import Flask, request, redirect, url_for, session, send_file
import boto3
import os
from io import BytesIO

app = Flask(__name__)
app.secret_key = "supersecretkey"

# ================= AWS CONFIG =================
BUCKET = "meharproject"

s3 = boto3.client(
    "s3",
    region_name="us-east-1"
)

# ================= LOGIN =================
@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]

        # Simple authentication (demo purpose)
        if username == "admin" and password == "admin123":
            session["user"] = username
            return redirect("/upload")
        else:
            return "Invalid credentials!"

    return '''
        <h2>Login</h2>
        <form method="post">
            Username: <input name="username"><br><br>
            Password: <input type="password" name="password"><br><br>
            <button type="submit">Login</button>
        </form>
    '''

# ================= UPLOAD =================
@app.route("/upload", methods=["GET", "POST"])
def upload():
    if "user" not in session:
        return redirect("/")

    if request.method == "POST":
        file = request.files["file"]
        s3.upload_fileobj(file, BUCKET, file.filename)
        return "File uploaded successfully! <br><a href='/files'>View Files</a>"

    return '''
        <h2>Upload File</h2>
        <form method="post" enctype="multipart/form-data">
            <input type="file" name="file">
            <button type="submit">Upload</button>
        </form>
        <br><a href="/files">View Files</a>
    '''

# ================= LIST FILES =================
@app.route("/files")
def list_files():
    if "user" not in session:
        return redirect("/")

    response = s3.list_objects_v2(Bucket=BUCKET)
    files = response.get("Contents", [])

    output = "<h2>Uploaded Files</h2>"

    for file in files:
        filename = file["Key"]
        output += f"""
```

**Fig2.-Web Interface Code**

```python
# ================= DOWNLOAD =================
@app.route("/download/<filename>")
def download_file(filename):
    if "user" not in session:
        return redirect("/")

    file_obj = s3.get_object(Bucket=BUCKET, Key=filename)
    return send_file(
        BytesIO(file_obj["Body"].read()),
        download_name=filename,
        as_attachment=True
    )

# ================= VERSIONING =================
@app.route("/versions/<filename>")
def file_versions(filename):
    if "user" not in session:
        return redirect("/")

    versions = s3.list_object_versions(Bucket=BUCKET, Prefix=filename)
    output = f"<h2>Versions of {filename}</h2>"

    for version in versions.get("Versions", []):
        version_id = version["VersionId"]
        output += f"""
        Version ID: {version_id}
        <a href='/download_version/{filename}/{version_id}'>Download</a>
        <br><br>
        """

    output += "<br><a href='/files'>Back</a>"
    return output

@app.route("/download_version/<filename>/<version_id>")
def download_version(filename, version_id):
    if "user" not in session:
        return redirect("/")

    file_obj = s3.get_object(
        Bucket=BUCKET,
        Key=filename,
        VersionId=version_id
    )

    return send_file(
        BytesIO(file_obj["Body"].read()),
        download_name=filename,
        as_attachment=True
    )

# ================= LOGOUT =================
@app.route("/logout")
def logout():
    session.pop("user", None)
    return redirect("/")

# ================= RUN =================
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

**Fig3.-Web Interface Code**

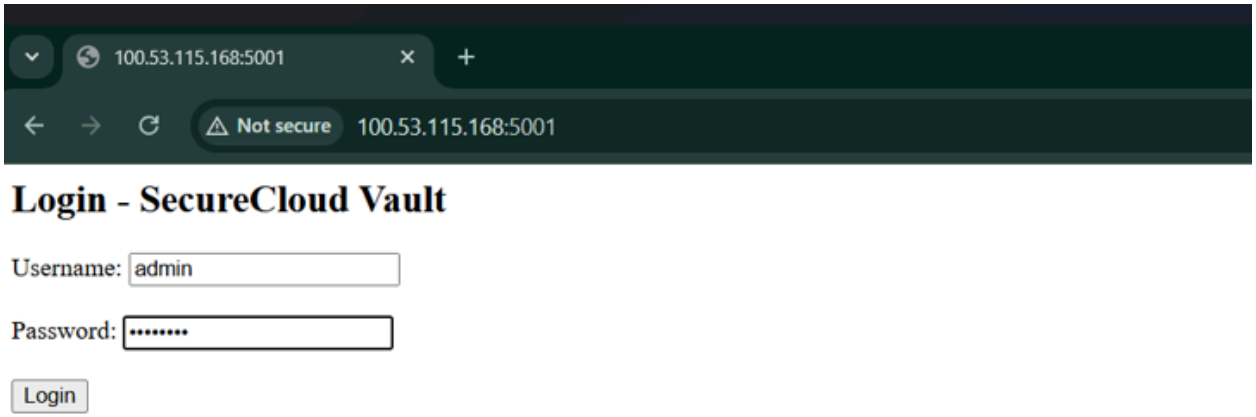


**Fig.4-Web Interface**

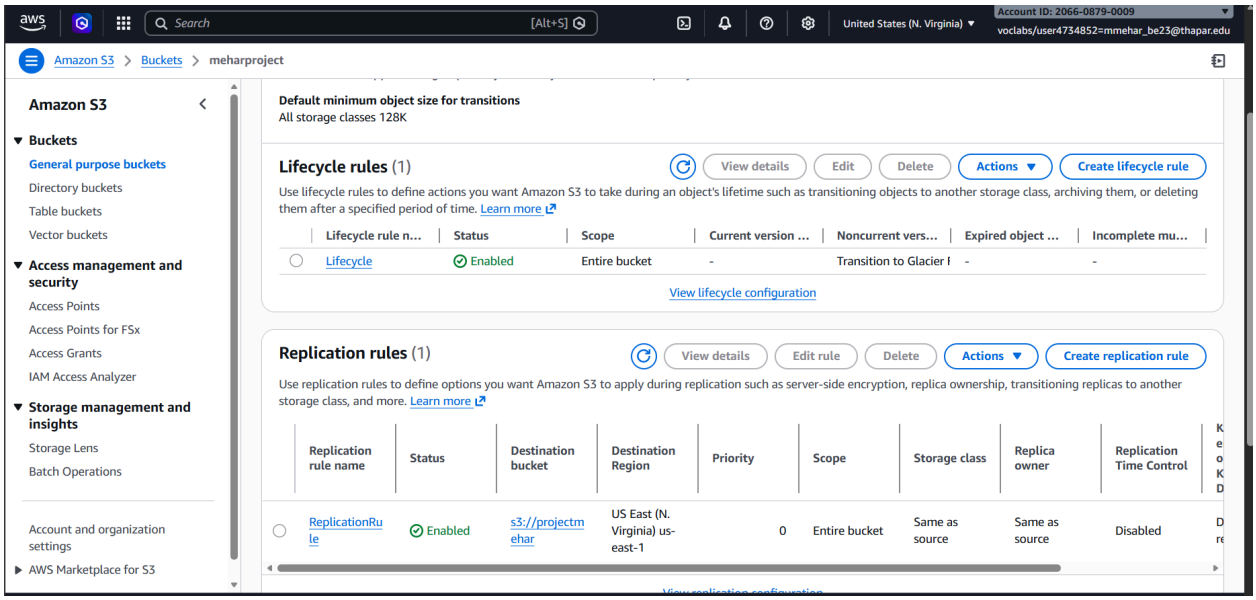


**Fig.5-Backup and Replication Rules**

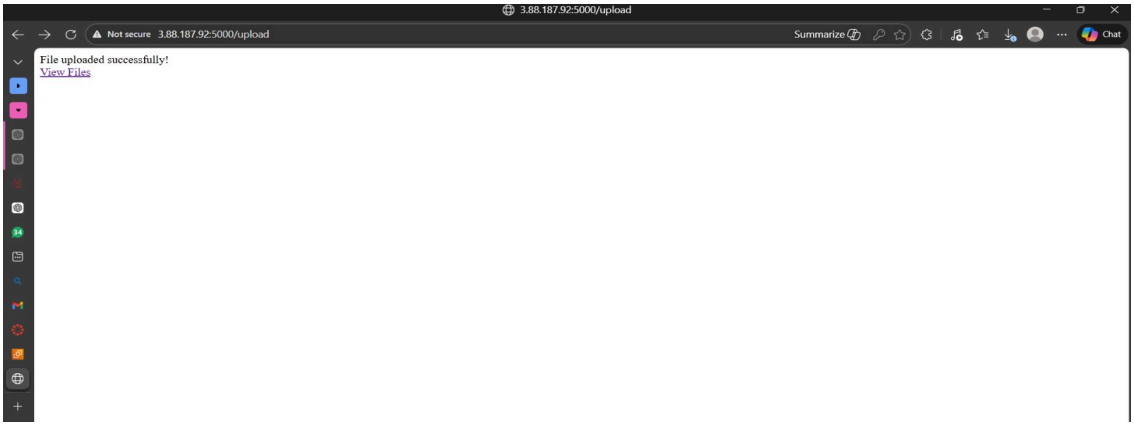


**Fig.6-File Uploaded**

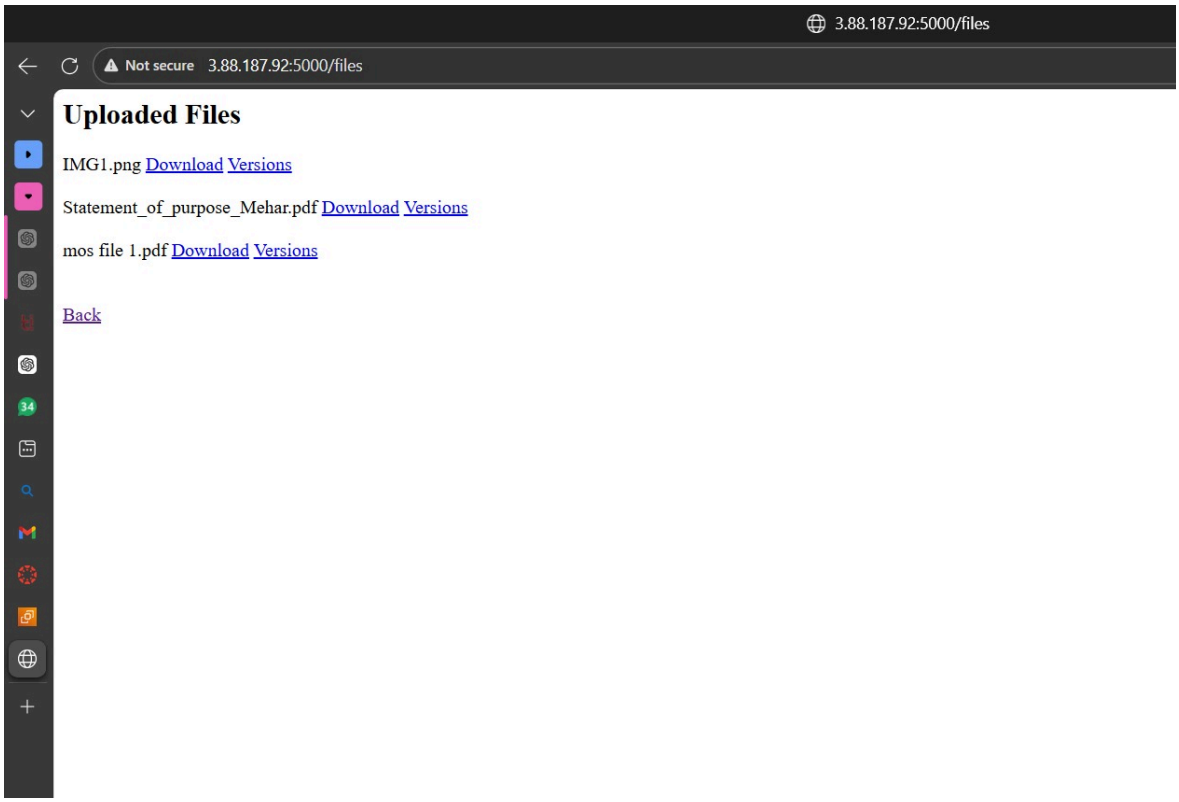


**Fig.7-List of Uploaded Files**

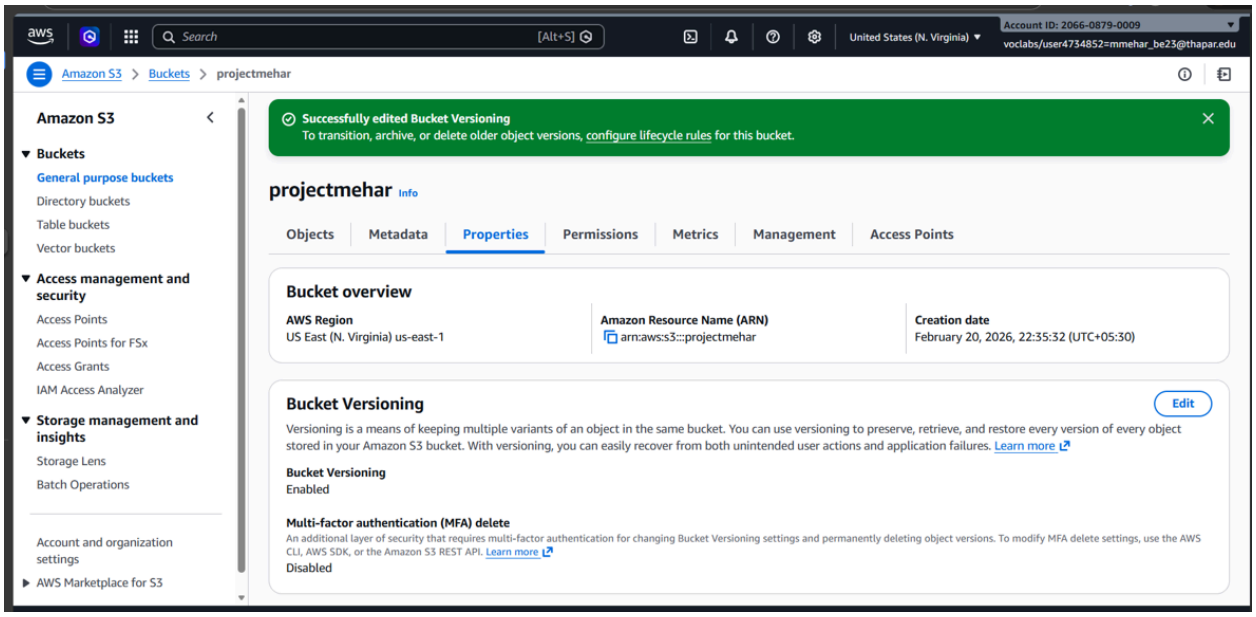


**Fig.8-Enabling the Version**

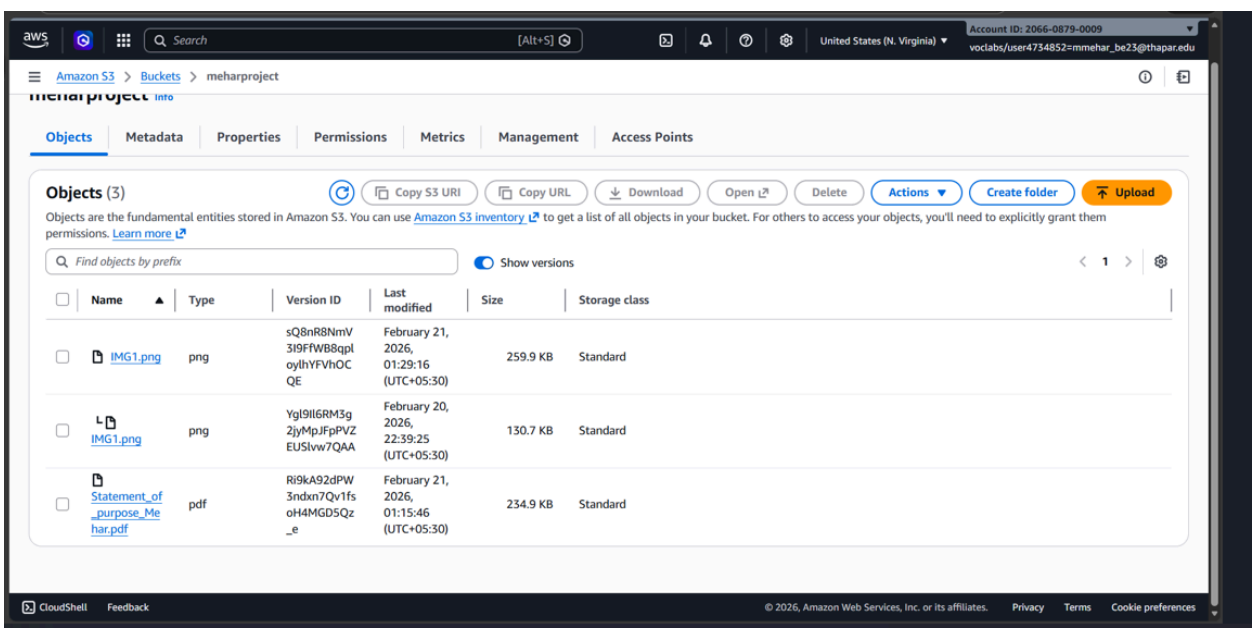


**Fig.9-Different Versions**

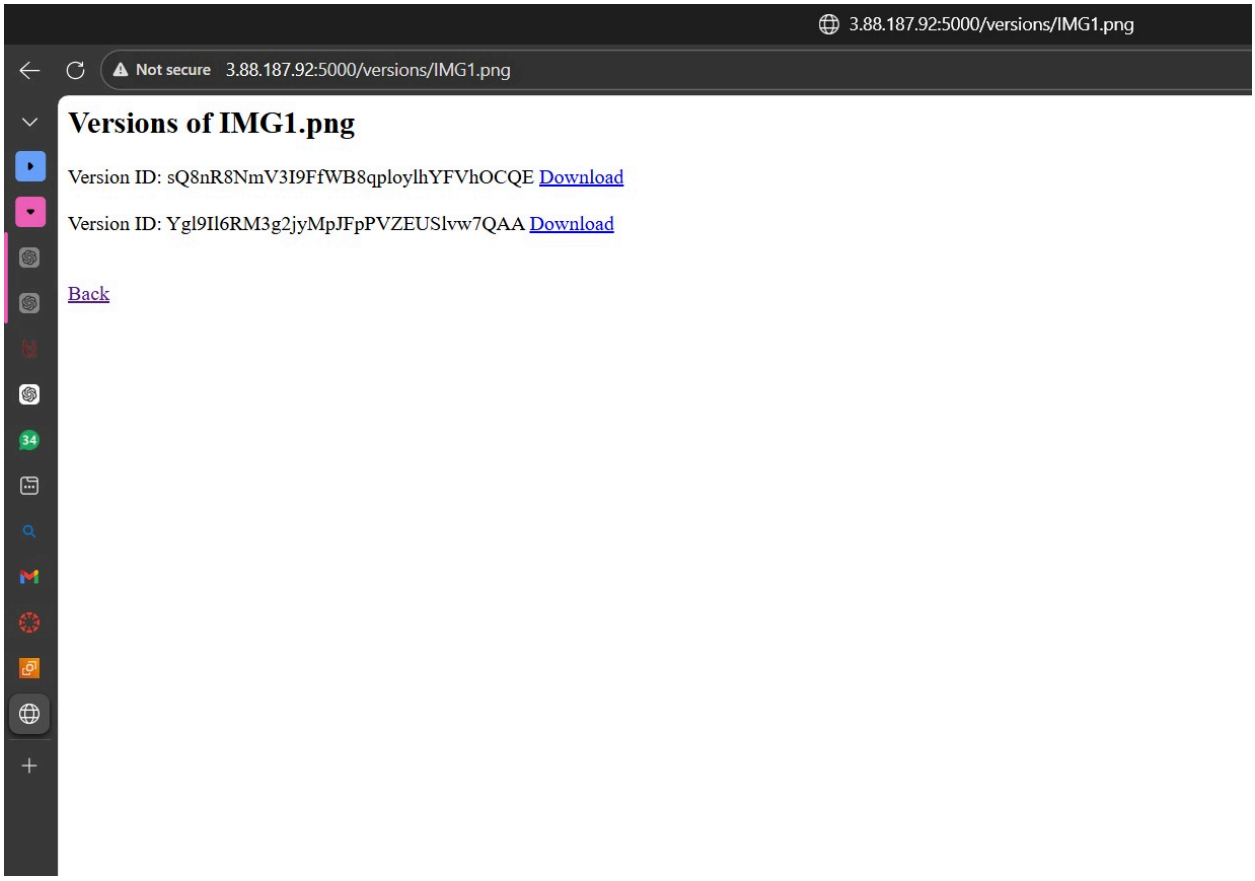


**Fig.10-Versions of IMG1**

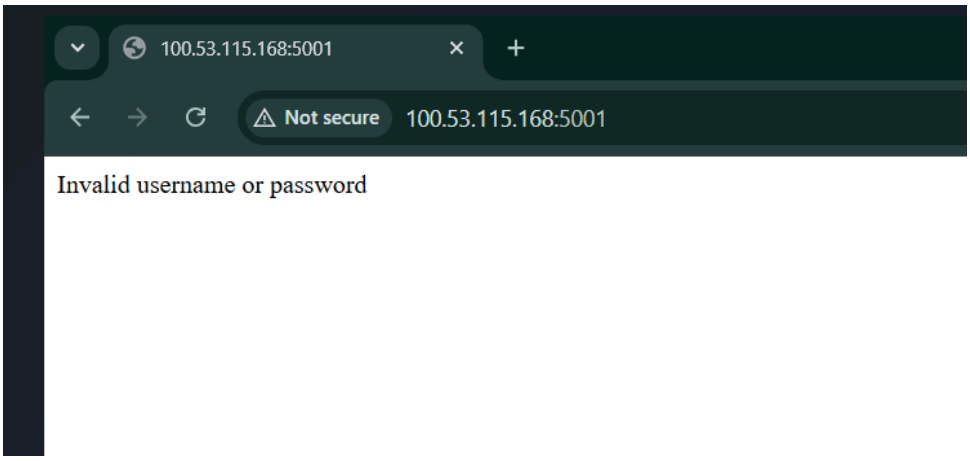


**Fig.11- User authentication Verified**

```python
from flask import Flask, request, redirect, url_for, session, send_file
import boto3
import os
app = Flask(__name__)
app.secret_key = "securecloudsecret"
s3 = boto3.client('s3')
BUCKET = "meharproject"
USERNAME = "admin"
PASSWORD = "admin123"
# ---------------- LOGIN ----------------
@app.route('/', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        if request.form['username'] == USERNAME and request.form['password'] == PASSWORD:
            session['user'] = USERNAME
            return redirect(url_for('upload'))
        else:
            return "Invalid username or password"

    return '''
    <h2>Login - SecureCloud Vault</h2>
    <form method="post">
        Username: <input type="text" name="username"><br><br>
        Password: <input type="password" name="password"><br><br>
        <input type="submit" value="Login">
    </form>
    '''
# ---------------- UPLOAD ----------------
@app.route('/upload', methods=['GET', 'POST'])
def upload():
    if 'user' not in session:
        return redirect(url_for('login'))

    if request.method == 'POST':
        file = request.files['file']
        s3.upload_fileobj(file, BUCKET, file.filename)
        return redirect(url_for('list_files'))

    return '''
    <h2>Upload File</h2>
    <form method="post" enctype="multipart/form-data">
        <input type="file" name="file">
        <input type="submit" value="Upload">
    </form>
    <br>
    <a href="/files">View Files</a>
    <br><br>
    <a href="/logout">Logout</a>
    '''
# ---------------- FILE LIST ----------------
@app.route('/files')
def list_files():
    if 'user' not in session:
        return redirect(url_for('login'))
    response = s3.list_objects_v2(Bucket=BUCKET)
    files = response.get('Contents', [])
    html = "<h2>Files (Latest Versions)</h2><ul>"
    for f in files:
        name = f['Key']
        html += f'<li>{name} - <a href="/versions/{name}">View Versions</a></li>'
    html += "</ul><br><a href='/upload'>Upload More</a>"
    html += "<br><br><a href='/logout'>Logout</a>"
    return html

# ---------------- VERSION LIST ----------------
@app.route('/versions/<filename>')
def list_versions(filename):
    if 'user' not in session:
        return redirect(url_for('login'))
    response = s3.list_object_versions(Bucket=BUCKET, Prefix=filename)
    versions = response.get('Versions', [])
    html = f"<h2>Versions of {filename}</h2><ul>"
    for v in versions:
```

**Fig.12-Deployment Code**

```python
# ---------------- VERSION LIST ----------------
@app.route('/versions/<filename>')
def list_versions(filename):
    if 'user' not in session:
        return redirect(url_for('login'))
    response = s3.list_object_versions(Bucket=BUCKET, Prefix=filename)
    versions = response.get('Versions', [])
    html = f"<h2>Versions of {filename}</h2><ul>"
    for v in versions:
        vid = v['VersionId']
        last = v['LastModified']
        size = v['Size']
        html += (
            f"<li>Version ID: {vid} | {last} | {size} bytes "
            f"- <a href='/download/{filename}/{vid}'>Download</a></li>"
        )
    html += "</ul><br><a href='/files'>Back to Files</a>"
    html += "<br><br><a href='/logout'>Logout</a>"
    return html
# ---------------- DOWNLOAD (VERSIONED) ----------------
@app.route('/download/<filename>/<version_id>')
def download_version(filename, version_id):
    if 'user' not in session:
        return redirect(url_for('login'))
    local_path = f"/tmp/{filename}"
    s3.download_file(BUCKET, filename, local_path, ExtraArgs={'VersionId': version_id})
    return send_file(local_path, as_attachment=True)
# ---------------- LOGOUT ----------------
@app.route('/logout')
def logout():
    session.pop('user', None)
    return redirect(url_for('login'))
# ---------------- MAIN ----------------
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

**Fig.13-Deployment Code Continued**

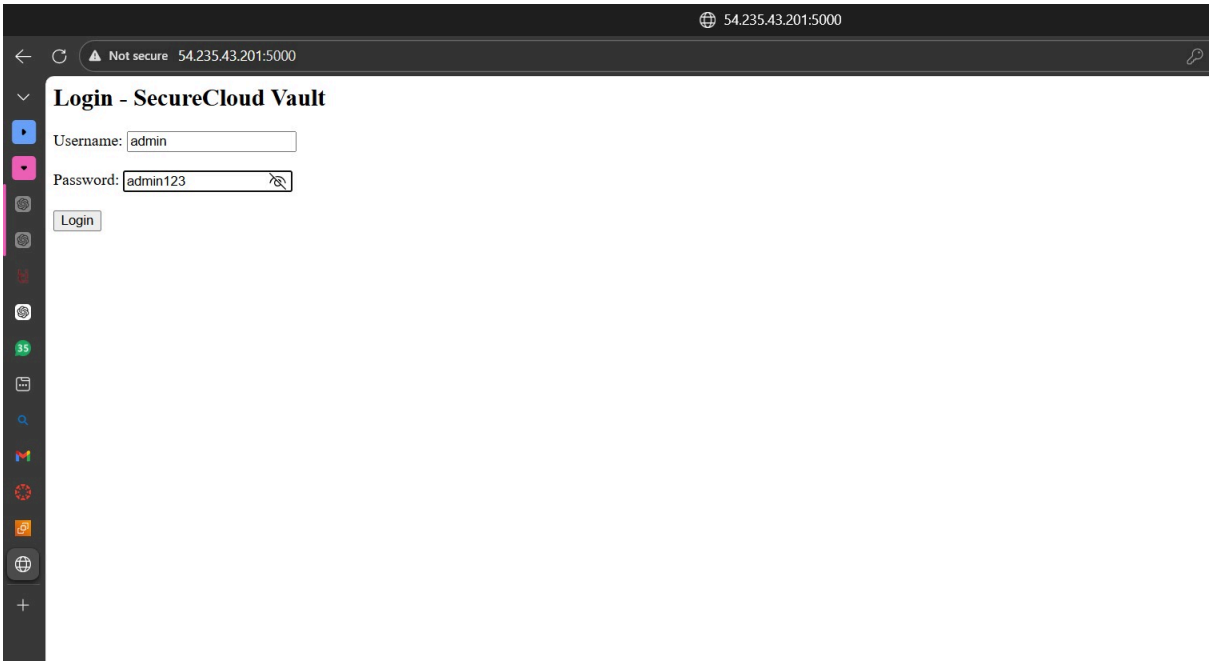


**Fig.14-Testing and Deployment(Login)**

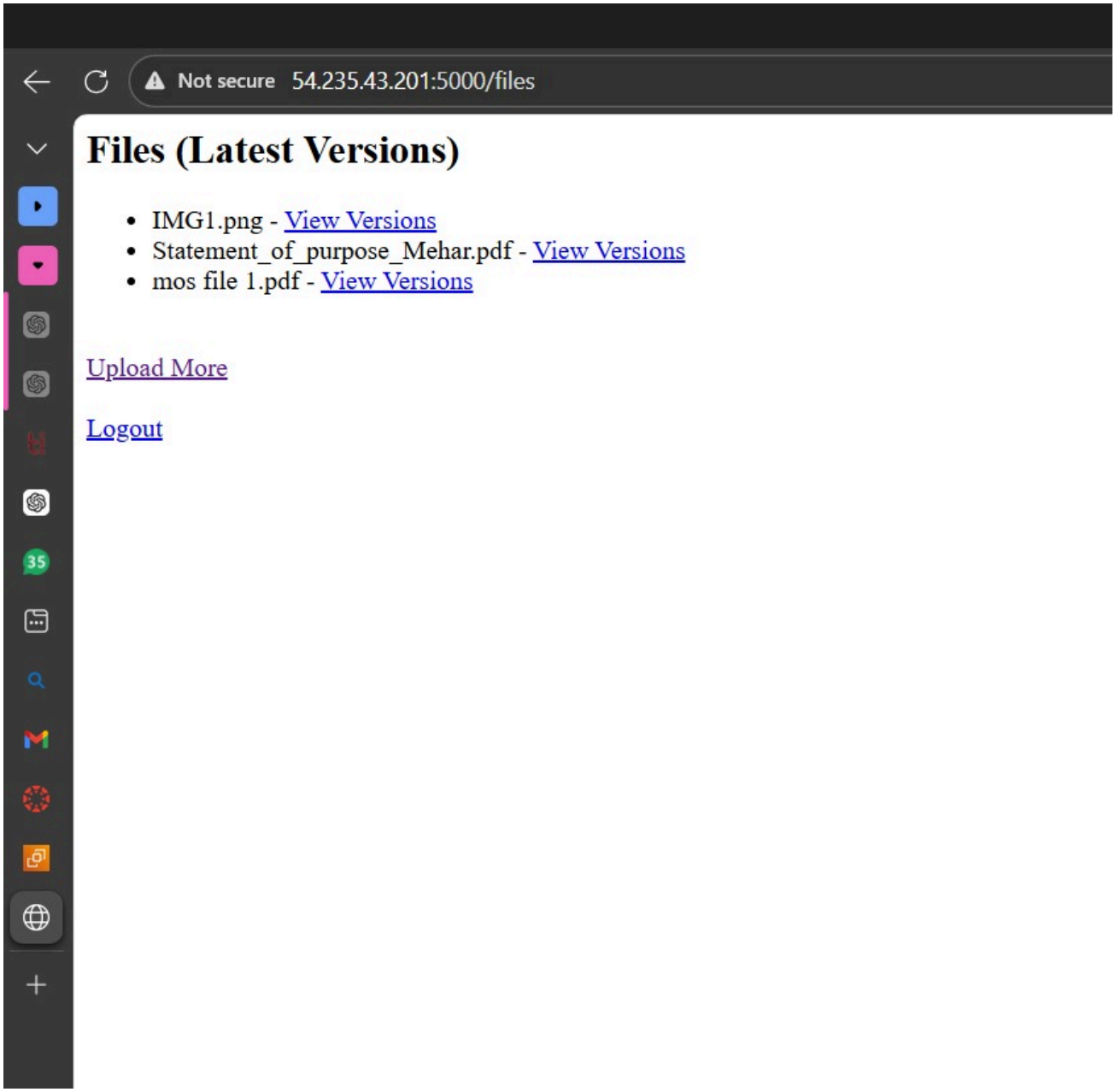


**Fig.15-Testing and Deployment(List of Files)**

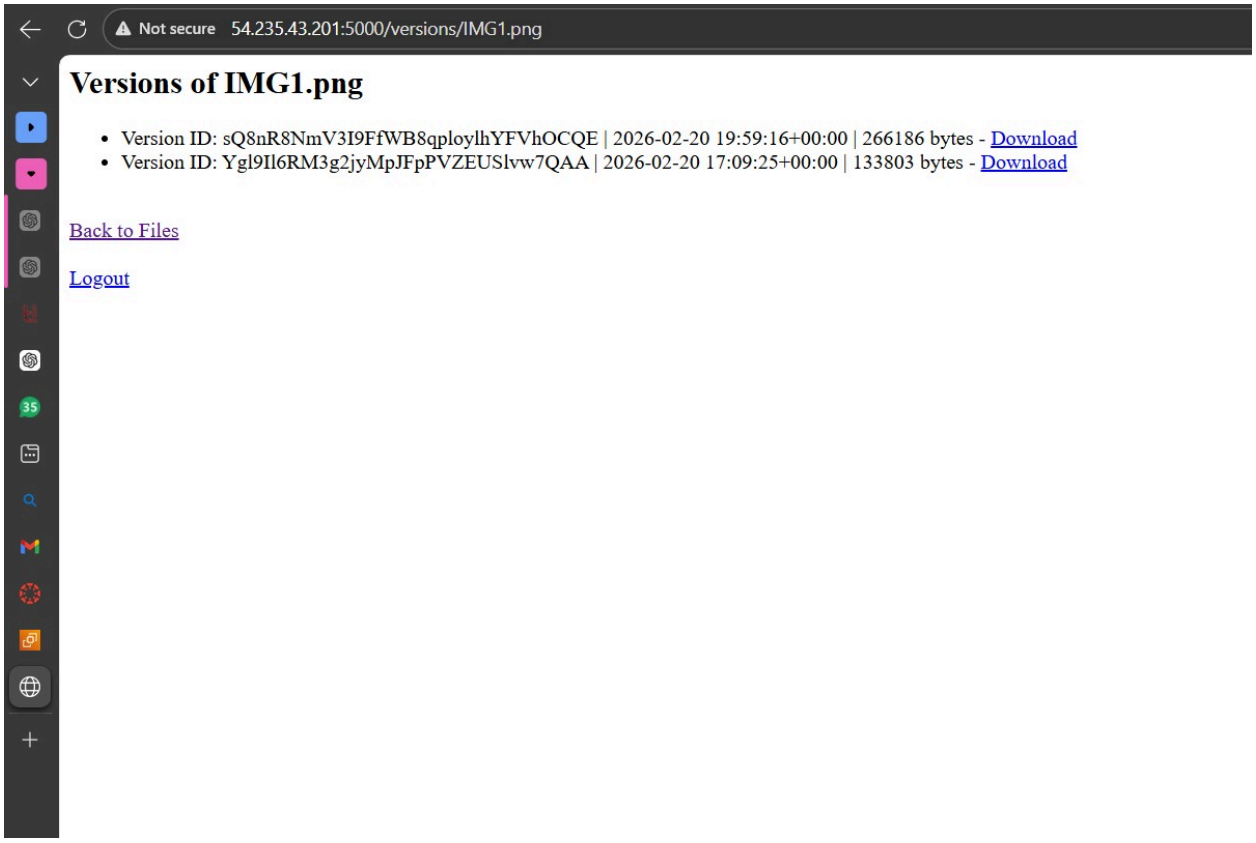


**Fig.16-Testing and Deployment(Versioning)**

## 9) RESULT:

After successful implementation:

- User logs in securely.
- Files upload to Main S3 bucket.
- Same file re-upload creates V1, V2, V3… (Versioning).
- Files automatically copy to Backup bucket using replication.
- Lifecycle rules manage older versions for cost and cleanup.
- User can download required file or specific versions.

Final Outcome: Secure cloud file system with IAM + backup + replication + version control.

## 10) ADVANTAGES:

- Secure data storage
- High availability
- Automatic backup
- Scalable architecture

- User authentication

**11)LIMITATIONS:**

- Internet dependency
- Basic authentication (not multi-user)
- No encryption customization

**12) CONCLUSION:**

This project successfully demonstrates a secure cloud storage application by integrating:

- IAM-based secure access
- Web-based file upload/download
- Automatic replication for backup
- Versioning for file history (V1, V2…)
- Lifecycle policies for storage optimization

Hence, the system provides high availability, disaster recovery, data integrity, and controlled access, meeting the objectives of a secure cloud application.

**13) FUTURE SCOPE:**

- Multi-user authentication
- File encryption
- Download functionality
- Activity logs
- UI enhancement

**14) REFERENCES:**


- AWS Documentation
- Flask Official Documentation
- Boto3 Documentation
- Cloud Computing Lecture Notes